

BASED ON BACHKATOR

Turn vague feature requests into polished product.

Describe the outcome. Bach coordinates the agents, reviews, and gates until it is ready to ship.

Enterprises do not need more demos. They need control.

Agents say “done”

But teams still need proof: tests, security, plan coverage, architecture compliance, and merge hygiene.

CI is too narrow

Traditional pipelines verify commands. They do not verify whether the agent fulfilled intent.

Governance is missing

Security and platform teams need standardized policies, audit trails, and cross-repo visibility.

Agents should not decide when they are done. Your company policy should.

Bachkator becomes the orchestration and evidence layer between natural-language software intent and production-ready code.

Self-improving spec development.

The spec is not a static document. Every agent attempt, reviewer finding, policy failure, and merge outcome feeds back into better prompts, stronger policies, and clearer plans.

Opinionated spec-driven development

- Plan-first workflow
- Built-in implementation, review, and merge lanes
- Default policies for tests, docs, security, architecture, and acceptance
- Automatic retries until the spec and policy pass

DIY enterprise mode

- Bring your own prompts and reviewer agents
- Import company policies from a central registry
- Plug in internal security, compliance, and architecture checks
- Tune thresholds by repo, team, risk level, or release type

Agents become a native delivery primitive.

```
agent "checkout_refactor" {  
  provider = opencode  
  role      = "implementer"  
  
  plan   = "plans/checkout.md"  
  branch = "agents/checkout"  
  policy = policy.merge  
  
  retry {  
    attempts = 5  
    until     = policy.passes  
  }  
}
```

What this buys

- Branch and workspace isolation
- Session resume and retry semantics
- Structured reports instead of magic log markers
- Audit-ready run history for every attempt

Every company brings its own definition of “ready”.

```
import "@acme/bach-policies/merge@v4"
import "@acme/bach-prompts/security@v2"

policy "merge" {
  type = "quality"

  reviewer "security" {
    provider = opencode
    prompt   = prompt.security_review
  }

  quality_gate {
    metric = "security.high.count"
    max    = 0
  }
}
```

Enterprise product surface

- Policy registry and versioning
- Prompt packages with approvals
- Reusable reviewer agents
- Compliance-grade gate history

Prompts become governed, reusable infrastructure.

```
prompt "plan_review" {  
  param "plan"  
  param "branch"  
  param "diff"  
  
  body = <<EOF  
Compare the branch against ${param.plan}.  
Find missing scope, tests, docs,  
and unresolved questions.  
Emit Bach quality findings only.  
EOF  
}
```

Why enterprises care

Security, architecture, compliance, and product teams can standardize how agents review work without touching application repos.

Every improvement loop leaves evidence.

Improvement loop

01

Spec

Turn the request into acceptance criteria, scope, risks, and tests.

02

Build

Implementation agents work in isolated branches and write reports.

03

Review

Policy reviewers produce normalized findings and gate results.

04

Improve

Agents fix unresolved findings until the merge policy passes.

OPEN FINDINGS

0 blocking

POLICY

merge@v8

DECISION

ready

ATTEMPT	OPEN	GATE	NEXT ACTION
Spec draft	7 findings	failed	Add acceptance criteria
Implementation	3 findings	failed	Fix tests and auth gap
Retry	0 findings	passed	Ready for merge queue

MERGE VERDICT

Ready

The branch has no blocking findings, required reviewers passed, and the evidence is stored for audit.

Policy

merge@v8

Reviewers

3 agents

Open blockers

0

Describe what you want. Get a production-ready branch in X time.

1

Describe

"Refactor checkout to support split shipments."

2

Plan

Bach creates or imports a scoped delivery plan.

3

Implement

Agent works in an isolated branch/workspace.

4

Review

Company reviewer agents produce findings.

5

Retry

Bach loops until policy passes or timebox expires.

Generate workflows that already know company rules.

CLI

```
bach generate workflow \  
  --feature "split shipments in checkout" \  
  --org applause \  
  --risk-profile payments \  
  --policy applause/merge@v8 \  
  --prompt-pack applause/platform@v12 \  
  --certify
```

Generated evidence

- Spec with acceptance criteria
- Bachfile agent workflow
- Required reviewer policy bindings
- Risk-specific test matrix
- Owner and approval requirements

Certification:

- ✓ uses applause/policy.merge@v8
- ✓ requires security_review@v4 for payments risk
- ✓ includes regression tests for partial fulfillment
- ✓ blocks merge until unresolved findings = 0

Generate an execution graph, not a pile of prompts.

```
bach generate batch \  
  --epic "checkout modernization" \  
  --feature "split shipments" \  
  --feature "refund retry ledger" \  
  --feature "fraud review queue" \  
  --parallelize safe \  
  --merge serialized \  
  --policy applause/merge@v8
```

```
group "implement_checkout_epic" {  
  targets = [  
    "agent/split_shipments",  
    "agent/refund_retry_ledger",  
    "agent/fraud_review_queue",  
  ]  
}  
  
pipeline "merge_checkout_epic" {  
  steps = [  
    "agent/merge_split_shipments",  
    "agent/merge_refund_retry_ledger",  
    "agent/merge_fraud_review_queue",  
    "group/gate",  
  ]  
}
```

The enterprise moat is the evidence graph.

Runs

Every agent attempt, reviewer pass, retry, merge, and gate result.

Findings

Normalized severity, file, line, rule, owner, resolved state, and evidence.

Policies

Which company policy version approved or rejected each branch.

Outcomes

Cycle time, failure modes, merge readiness, escaped defects, and ROI.

OSS can run a workflow locally. Enterprise learns across every repo, spec, prompt, policy, reviewer, model, and delivery outcome.

Enterprise value is policy, memory, and certification.

```
enterprise "applause" {  
  sync = true  
  
  registries {  
    policies = "applause/policies"  
    prompts  = "applause/prompts"  
    findings = "applause/evidence"  
  }  
  
  enforce {  
    require_certified_workflows = true  
    require_policy_version      = true  
    block_unapproved_prompts    = true  
  }  
}
```

Why it compounds

- Every failed spec improves the next spec template
- Every reviewer finding tunes prompt packs
- Every policy miss becomes a policy recommendation
- Every accepted branch creates benchmark data

What makes it an enterprise product.

Governance

- SSO/RBAC
- Prompt approvals
- Policy change control
- Audit exports

Operations

- Central runner fleet
- Provider credentials
- Cost controls
- Workspace lifecycle

Delivery

- GitHub/GitLab checks
- Reviewer bots
- Merge queues
- Release evidence

Different buyers, one story: trust the process.

VP Engineering

"Increase throughput without lowering the bar for merge quality."

Platform

"Standardize agent workflows across every repo and team."

Security

"No agent-written code merges without review findings and gates."

Open-source wedge. Enterprise control plane.

OSS Bach

Local CLI, Bachfile targets, quality gates, reports, agent primitives.

Team

Shared policies, hosted dashboards, GitHub checks, prompt registry.

Enterprise

SSO, RBAC, private runners, audit exports, compliance workflows, org analytics.

PRODUCT PROMISE

AI agents can write code. Bach decides whether it is shippable.

Make agentic software delivery observable, policy-gated, and enterprise-safe.